

Машинное обучение и глубокое обучение: сравнительное исследование

Автор: [ФИО студента]

Группа: [ProjectGroup\_ID]

Университет: [Название]

Дисциплина: Machine Learning & Deep Learning

Дата: Май 2026

#### Аннотация

В данной работе представлено полное сквозное исследование, объединяющее классические методы машинного обучения, методы снижения размерности и глубокие нейронные сети. На табличных наборах данных UCI Wine Quality и Covertype реализован воспроизводимый конвейер с предобработкой признаков, стратифицированным разбиением 70/15/15, подбором гиперпараметров и оценкой по метрикам accuracy, precision, recall, F1 и ROC-AUC. Для визуализации и анализа структуры данных применены PCA, t-SNE, UMAP и LDA; исследовано переобучение лучшего классификатора на PCA-признаках с сохранением 95% дисперсии. В блоке глубокого обучения обучены многослойный перцептрон на Fashion-MNIST, свёрточная сеть с нуля на CIFAR-10 и выполнена трансферная дообучение ResNet18. Сравнительный анализ показывает компромисс между точностью, временем обучения, размером модели и интерпретируемостью. Все эксперименты воспроизводимы: зафиксированы random seed, логирование и сохранение артефактов в каталоге results/. Отчёт охватывает шесть учебных задач и сопровождается Jupyter-блокнотами, исходным кодом и демонстрационным скриптом.

Ключевые слова: классификация, ансамбли, PCA, свёрточные нейронные сети, трансферное обучение, кросс-валидация.

## **Задача 1. Теоретические основы классических алгоритмов обучения**

### **1.1 Введение в постановку задачи**

Задача обучения с учителем формулируется как поиск отображения  $f: \mathcal{X} \rightarrow \mathcal{Y}$  по обучающей выборке  $\{(x_i, y_i)\}_{i=1}^n$ , минимизирующего ожидаемый риск. В классификации  $\mathcal{Y}$  — конечное множество меток. Качество оценивается функцией потерь и обобщающей способностью на тестовой выборке, не участвовавшей в обучении. Классические алгоритмы различаются индуктивными предпосылками: линейные модели предполагают разделимость гиперплоскостью, деревья — кусочно-постоянные аппроксимации, метод опорных векторов — максимизацию зазора, k-NN — локальную гладкость в пространстве признаков. Понимание этих предпосылок критично для выбора модели и диагностики переобучения и недообучения.

## 1.2 k-ближайших соседей (k-NN)

Метод k-NN относится к ленивому (instance-based) обучению: фаза «обучения» сводится к запоминанию обучающих объектов. Для запроса  $x$  вычисляются расстояния  $d(x, x_i)$  до всех обучающих точек (часто евклидово или по Минковскому), выбираются  $k$  объектов с наименьшим расстоянием, и предсказание — мажоритарный класс среди их меток. При весах, обратно пропорциональных расстоянию, вклад ближних соседей усиливается. Математически:  $\hat{y} = \mathop{\mathrm{mode}}\limits_{y_i : x_i \in N_k(x)}$ . Сложность обучения  $O(1)$ , инференса  $O(nd)$  на выборке из  $n$  объектов и  $d$  признаков — узкое место на больших данных. Преимущества: простота, отсутствие явного обучения, естественная мультиклассовость. Ограничения: чувствительность к масштабу признаков (необходима нормализация), проклятие размерности, шумные метки соседей. Гиперпараметр  $k$  управляет смещением-дисперсией: малый  $k$  — высокая дисперсия, большой — сглаживание границ. Лучшие сценарии: небольшие и средние табличные данные после стандартизации, базовые линии.

## 1.3 Решающие деревья и случайный лес

Дерево решений рекурсивно делит пространство признаков по правилам  $x_j \leq t$ , минимизируя критерий неоднородности (индекс Джини  $G = 1 - \sum_k p_k^2$ ) или энтропия). Лист предсказывает мажоритарный класс. Интерпретируемость высока, но одно дерево склонно к переобучению. Случайный лес (Random Forest) строит ансамбль из  $T$  деревьев на бутстрап-подвыборках с случайным подмножеством признаков на каждом разбиении; итоговое решение — голосование. Это снижает дисперсию при сохранении низкого смещения отдельных деревьев. Обучение масштабируется как  $O(T \cdot n \log n \cdot d)$  при жадных сплитах. Гиперпараметры: `max_depth`, `min_samples_split`, `n_estimators`, `max_features`. Лес устойчив к выбросам и смешанным типам признаков после кодирования; важность признаков доступна через среднее уменьшение неоднородности. Ограничения: большой размер модели, слабее интерпретируемость, чем у одного дерева.

## 1.4 Метод опорных векторов (SVM)

SVM ищет разделяющую гиперплоскость  $(w^{\top} x + b = 0)$  с максимальным зазором между классами. Для линейно неразделимых данных используется ядерный трюк:  $K(x_i, x_j)$  неявно отображает точки в пространство большей размерности. Оптимизация — квадратичная задача с ограничениями; решение зависит только от опорных векторов. Решающая функция:  $f(x) = \mathop{\mathrm{sign}}\limits(\sum_i \alpha_i y_i K(x_i, x) + b)$ . Параметр  $C$  балансирует ширину зазора и штраф за ошибки;  $\gamma$  в

RBF-ядре задаёт «радиус» влияния. Преимущества: эффективность в средних размерностях, сильная теория обобщения. Недостатки: квадратичная/кубическая сложность по  $(n)$  для точных солверов, необходимость масштабирования признаков, калибровка вероятностей отдельно (Platt scaling). Хорошо подходит для табличных данных среднего объёма и текстов с линейным ядром.

## 1.5 Логистическая регрессия

Логистическая регрессия моделирует апостериорную вероятность класса через сигмоиду (бинарный случай) или softmax (мультикласс):  $P(y=k|x) = \mathrm{softmax}(Wx)_k$ . Обучение — максимизация лог-правдоподобия (кросс-энтропия) с возможной L2-регуляризацией  $\frac{\lambda}{2} \|W\|^2$ . Функция выпуклая; оптимизация градиентными методами (L-BFGS, SAGA). Правило решения:  $\hat{y} = \arg\max_k P(y=k|x)$ . Сложность обучения линейна по числу итераций и признаков; инференс —  $O(d)$ . Преимущества: скорость, калиброванные вероятности, интерпретируемые веса. Недостатки: линейная граница в исходных признаках без feature engineering. Является сильным базовым методом и компонентом стекинга.

## 1.6 Наивный байесовский классификатор

Наивный Байес предполагает условную независимость признаков при фиксированном классе:  $P(y|x) \propto P(y) \prod_j P(x_j|y)$ . Для непрерывных признаков — гауссовы плотности; для счётных — мультиномиальное распределение. Обучение сводится к оценке частот и параметров по классам —  $O(nd)$ . Инференс быстрый. Предпосылка независимости редко выполняется, но классификатор устойчив на текстовых данных (мешок слов) и служит конкурентным baseline. Сглаживание Лапласа предотвращает нулевые вероятности.

## 1.7 Сравнительная таблица и вывод по задаче 1

Вывод: Выбор алгоритма должен опираться на размер данных, размерность, требования к вероятностям и интерпретируемости. В последующих задачах проекта на табличных данных сравниваются четыре метода: логистическая регрессия, случайный лес, SVM и k-NN с единым конвейером предобработки.

## Задача 2. Классический конвейер машинного обучения

### 2.1 Постановка и наборы данных

Во второй задаче реализован полный конвейер supervised classification на двух наборах данных, соответствующих требованиям задания. Набор А — Wine Quality (UCI, OpenML id=287): табличные физико-химические признаки вин и целевая переменная — оценка качества (мультиклассовая, дискретные уровни). Набор В — Coverture: лесной покров, более 580 тысяч записей в полной версии; для управляемого времени обучения использована стратифицированная подвыборка 15 000 объектов и 7 классов типов лесного покрова, что удовлетворяет требованию «мультикласс  $\geq 1000$  образцов». Оба набора отражают реальные прикладные сценарии: контроль качества продукции и дистанционное зондирование/экология. Перед моделированием проведён анализ дисбаланса классов: при доле миноритарного класса ниже порога 0.3 активируется `class_weight='balanced'` в поддерживающих это моделях.

### 2.2 Предобработка и разбиение данных

Предобработка оформлена как `sklearn.pipeline.Pipeline` с `ColumnTransformer`. Числовые признаки проходят импутацию медианой и `StandardScaler` (нулевое среднее, единичная дисперсия). Категориальные — заполнение модой и `OneHotEncoder` с `handle_unknown='ignore'`. Такой дизайн предотвращает утечку информации из теста: все статистики `imputation` и `scaling` оцениваются только на обучающей части внутри кросс-валидации и финального `fit`. Разбиение выполнено стратифицированно в пропорции 70% train / 15% validation / 15% test. Для Wine Quality при редких классах стратификация автоматически отключается на втором этапе `split`, если в подвыборке остаётся менее двух объектов класса — это технически необходимое исключение, задокументированное в коде `split_train_val_test`.

### 2.3 Обучаемые классификаторы и подбор гиперпараметров

Обучены четыре классификатора: Logistic Regression (L2, солверы `lbfgs/saga`), Random Forest, SVM с RBF/линейным ядром и `probability=True` для ROC-AUC, k-NN с вариантами весов `uniform/distance`. Подбор гиперпараметров: `GridSearchCV` для LR, RF, k-NN; `RandomizedSearchCV` для SVM из-за большего пространства поиска. Метрика отбора — `weighted F1`, учитывающая дисбаланс через усреднение по классам с весами поддержки. Кросс-валидация — 5-fold stratified. Лучшие параметры и оценки сохраняются в `results/metrics/` и модели в `results/models/classical_ml/{dataset}/`.

### 2.4 Метрики оценки и визуализация

Для каждой модели на тесте вычислены: `accuracy`, `precision`, `recall`, `F1 (weighted)`, ROC-AUC (`one-vs-rest` для мультикласса), матрицы ошибок и отчёты классификации. Дополнительно проведена кросс-валидация на объединении `train+val` с отчётом `mean  $\pm$  std`. Построены ROC-кривые по классам, тепловые карты `confusion matrix`, столбчатые диаграммы сравнения моделей. Для интерпретируемости обучено решающее дерево глубины 3 на преобразованных признаках и визуализирована структура разбиений (`decision_tree_depth3.png`). Результаты на Coverture (быстрый прогон): лучший `F1` у k-NN; на Wine Quality — Random Forest, что согласуется с теорией: ансамбли деревьев хорошо улавливают нелинейные взаимодействия химических признаков.

## 2.5 Переобучение и недообучение

На валидационных кривых Random Forest при чрезмерной глубине наблюдается рост разрыва train–validation — признак переобучения. Логистическая регрессия демонстрирует более высокое смещение на сложном Covertype, но устойчивую генерализацию. Регуляризация через  $C$  в LR и SVM, ограничение `max_depth` в RF и выбор  $k$  в  $k$ -NN — основные рычаги контроля сложности. Ручной численный пример  $k$ -NN (см. `docs/theory/manual_knn_example.md`) иллюстрирует вычисление евклидовых расстояний и мажоритарное голосование.

## 2.6 Вывод по задаче 2

Реализован воспроизводимый модульный конвейер (`src/data`, `src/classical_ml`, `scripts/run_classical_ml.py`), соответствующий лучшим практикам MLOps на уровне учебного проекта: конфигурация YAML, логирование, фиксированные `seed`. Jupyter-блокнот `notebooks/02_classical_ml_pipeline.ipynb` дублирует эксперимент для интерактивной демонстрации. Классические методы на табличных данных достигают конкурентной точности при умеренных вычислительных затратах и остаются обязательным этапом перед переходом к глубокому обучению.

## Задача 3. Снижение размерности и анализ представлений

### 3.1 Мотивация и постановка

Высокая размерность признакового пространства увеличивает риск переобучения, ухудшает интерпретацию и замедляет алгоритмы, чувствительные к расстоянию (k-NN, RBF-SVM). Методы снижения размерности позволяют выявить латентную структуру данных, визуализировать кластеры классов и, в случае PCA, сжать признаки с контролируемой потерей дисперсии. В задаче 3 на преобразованных признаках Covertypes применены PCA, t-SNE, UMAP и LDA; построены scree plot и двумерные вложения с раскраской по классам. Отдельно исследовано переобучение Random Forest на PCA-представлении, сохраняющем 95% объяснённой дисперсии, с количественным сравнением метрик до и после сжатия.

### 3.2 PCA: теория и реализация

Метод главных компонент ищет ортонормированные направления максимальной дисперсии. Первая компонента  $(w_1)$  решает  $\max_{\|w\|=1} \mathrm{Var}(Xw)$ ; последующие ортогональны предыдущим. Эквивалентно — разложение по собственным векторам ковариационной матрицы. Объяснённая дисперсия компоненты  $(k)$ :  $\lambda_k / \sum_j \lambda_j$ . Порог 95% задаёт минимальное  $(k)$ , при котором кумулятивная сумма  $\geq 0.95$ . В проекте построены scree plot и кумулятивная кривая (results/plots/dimensionality/pca\_explained\_variance.png). PCA — линейный метод: нелинейные многообразия могут плохо проецироваться, зато инференс быстрый (матричное умножение) и подходит как этап production-pipeline.

### 3.3 t-SNE и UMAP: нелинейные вложения

t-SNE минимизирует расхождение распределений парных расстояний в исходном и низкоразмерном пространстве, акцентируя локальную структуру. Параметр perplexity контролирует эффективное число соседей. Метод стохастичен, не сохраняет глобальные расстояния, не предназначен для прямого применения к новым точкам без переобучения embedding. UMAP строится на теории римановых многообразий и графов ближайших соседей; часто быстрее t-SNE на больших  $(n)$  и лучше сохраняет глобальную топологию. В проекте на подвыборке 2000 объектов получены 2D карты tsne\_2d.png и umap\_2d.png: видны частичные разделения классов Covertypes, перекрытия отражают сложность границ в исходном пространстве.

### 3.4 LDA: дискриминантный анализ

Linear Discriminant Analysis ищет проекцию, максимизирующую отношение межклассовой и внутриклассовой дисперсии (критерий Фишера). Для  $(C)$  классов существует не более  $(C-1)$  значимых дискриминантных направлений. LDA супервизорный — использует метки, в отличие от PCA. На Covertypes 2D-проекция LDA (lda\_2d.png) часто даёт более чёткое разделение кластеров, чем PCA, когда классы гауссовы в признаках. Ограничение: линейность границ в пространстве компонент.

### 3.5 Эксперимент PCA 95% и переобучение классификатора

После ColumnTransformer размерность признаков Covertypes составляет десятки/сотни измерений. PCA

с порогом 95% дисперсии сокращает их до меньшего  $\backslash(k\backslash)$  (значение фиксируется в `results/metrics/pca_comparison.json`). Random Forest с теми же гиперпараметрами обучен на полных и PCA-признаках; на тесте сравниваются accuracy и F1. Интерпретация: если accuracy падает незначительно, сжатие отбрасывает в основном шум; сильное падение указывает на потерю дискриминативных направлений, не совпадающих с осями максимальной дисперсии. Компромисс: скорость инференса и интерпретируемость (меньше признаков) против предельной точности.

### 3.6 Вывод по задаче 3

Визуализации подтверждают, что классы Covertypes частично разделимы, но с существенными перекрытиями — объясняет необходимость нелинейных моделей (RF, SVM-RBF). Для отчётности и презентации UMAP/t-SNE наиболее наглядны; для деплоя предпочтительнее PCA или LDA с валидацией на downstream-задаче. Блокнот `notebooks/03_dimensionality_reduction.ipynb` воспроизводит пайплайн; код — `scripts/run_dimensionality.py` и модуль `src/dimensionality/`.

## Задача 4. Многослойный перцептрон (MLP) на Fashion-MNIST

### 4.1 Выбор данных и архитектуры

Fashion-MNIST — 70 000 изображений предметов одежды 28×28 в градациях серого, 10 классов. Датасет сложнее MNIST за счёт визуальной схожести классов (рубашка vs куртка vs свитер), что делает его стандартным бенчмарком для полносвязных сетей перед переходом к CNN. Изображения вытянуты в векторы длины 784 и нормированы в  $[0, 1]$ . Архитектура MLP в проекте: вход 784 → скрытые слои 512 → 256 → 128 с Batch Normalization, активацией (ReLU / tanh / sigmoid в экспериментах), Dropout  $p=0.3$  → выход softmax на 10 классов. Таким образом выполнено требование «не менее трёх скрытых слоёв», dropout, batch norm и softmax.

### 4.2 Функция потерь и оптимизация

Используется sparse categorical crossentropy при целочисленных метках. Сравниваются три оптимизатора: SGD с momentum 0.9, Adam, RMSprop — с learning rate из конфигурации (0.01 для SGD, 0.001 для адаптивных). Early stopping по val\_loss с восстановлением лучших весов предотвращает переобучение. ReduceLROnPlateau снижает learning rate при плато валидации — практическая форма scheduling. Обучение 30 эпох (в демо-режиме --quick сокращено для CI). Кривые loss и accuracy сохранены в results/plots/mlp/ и results/metrics/mlp\_histories.json.

### 4.3 Теоретические аспекты

Исчезающие градиенты: при сигмоиде/tanh глубокие слои получают малые производные, обучение замедляется; ReLU частично устраняет проблему, но возможен «dying ReLU». Batch normalization нормализует мини-батч, вводя обучаемые  $(\gamma, \beta)$ ; стабилизирует распределение активаций и позволяет большие learning rate. Learning rate: слишком большой — расходимость; слишком малый — медленная сходимость и застревание в плато. Адаптивные методы (Adam) оценивают per-parameter шаги по оценкам первого и второго момента градиента. Регуляризация: dropout аппроксимирует ансамблирование; L2 на весах возможна через kernel\_regularizer (в расширенных экспериментах).

### 4.4 Сравнение активаций и оптимизаторов

Эксперименты показывают, что ReLU + Adam обычно даёт наиболее быструю сходимость и высокую validation accuracy. Sigmoid в скрытых слоях уступает по скорости и финальной метрике. SGD с правильным lr может сходиться к сильным минимумам, но требует тщательного подбора. График optimizer\_comparison.png сравнивает val\_loss и val\_accuracy по эпохам — наглядный материал для раздела «Discussion» отчёта.

### 4.5 Ограничения MLP на изображениях

MLP не учитывает пространственную локальность пикселей: перестановка пикселей внутри изображения разрушает семантику, но не меняет размерность вектора — для модели это разные объекты. Поэтому на Fashion-MNIST CNN (задача 5) ожидаемо превосходит MLP при том же бюджете параметров. MLP остаётся важным базисом для табличных и уже извлечённых признаков.

## 4.6 Вывод по задаче 4

Реализован модуль `src/deep_learning/mlp.py`, скрипт `scripts/run_mlp.py`, блокнот `notebooks/04_mlp_experiments.ipynb`. Достигнута рабочая accuracy на уровне типичных учебных реализаций; полные 30 эпох рекомендуются для финальной сдачи. Результаты демонстрируют влияние оптимизатора, `scheduling` и регуляризации на обобщающую способность — ключевые компетенции курса по глубокому обучению.

## Задача 5. Свёрточные нейронные сети и трансферное обучение

### 5.1 CNN с нуля на CIFAR-10

CIFAR-10 содержит 60 000 цветных изображений  $32 \times 32$ , 10 классов (транспорт, животные). Реализована сеть с тремя свёрточными блоками: в каждом два Conv2D  $3 \times 3$ , BatchNorm, ReLU, MaxPooling  $2 \times 2$ , Dropout 0.25. После блоков — GlobalAveragePooling, полносвязный слой 256, BatchNorm, Dropout 0.5, softmax. Число параметров  $\approx 325\,418$  (см. results/metrics/cnn\_metrics.json). Аугментация данных: горизонтальный flip, случайные сдвиги, zoom, jitter контраста — снижает переобучение и имитирует естественную вариативность сцены.

Обучение: Adam,  $\text{lr}=0.001$ , 30 эпох (требование задания), early stopping, ReduceLROnPlateau, чекпоинты в results/models/cnn\_scratch/. Итоговая модель — results/models/cnn/cifar10\_cnn\_final.keras. На тесте (быстрый прогон 3 эпохи  $\sim 62\%$ ; полное обучение 30 эпох ожидаемо 70–75%+ на CPU/GPU). Построены кривые обучения и confusion matrix с per-class метриками.

### 5.2 Архитектурная схема и параметры

Свёртки извлекают локальные паттерны (края, текстуры); глубина увеличивает рецептивное поле. MaxPooling обеспечивает инвариантность к малым сдвигам. BatchNorm ускоряет сходимость CNN так же, как в MLP. Схема архитектуры сохраняется в results/plots/cnn/architecture.png (или текстовый summary при отсутствии graphviz). Сравнение с MLP: CNN явно использует 2D-структуру, что критично для изображений.

### 5.3 Трансферное обучение ResNet18

Для эффективного использования предобученных представлений реализован двухэтапный fine-tuning на CIFAR-10 (изображения ресайзятся до  $224 \times 224$ , нормализация ImageNet):

1. Freeze backbone — заморожены все слои, кроме головы; обучается только линейный классификатор на 10 классов ( $\sim 10$  эпох).
2. Unfreeze — разморозка всей сети; дообучение с меньшим lr ( $\sim 20$  эпох).

Метрики (results/metrics/transfer\_learning\_metrics.json): test accuracy  $\sim 86\%$  (подвыборка train 10k для ускорения), 11.2M параметров, время этапов head / finetune зафиксировано. ResNet18 с остаточными связями облегчает градиентный поток в глубоких сетях.

### 5.4 Интерпретируемость

Модуль src/deep\_learning/interpretability.py поддерживает Grad-CAM: взвешенная сумма карт активаций последних свёрточных слоёв по градиентам целевого класса. Дополнительно возможны визуализации фильтров первого слоя и сетка «верно / неверно» с confidence (plot\_predictions\_grid). Это связывает практику с требованием объяснимости решений.

### 5.5 Сравнение CNN scratch vs transfer

### 5.6 Вывод по задаче 5

Блокноты 05\_cnn\_from\_scratch.ipynb и 06\_transfer\_learning.ipynb документируют эксперименты. GPU ускоряет обучение на 30 эпохах; в коде включён memory growth для TensorFlow. Трансферное обучение — промышленный стандарт при дефиците размеченных изображений; CNN с нуля — обязательный учебный этап для понимания свёрток и аугментации.

## Задача 6. Обзор продвинутых архитектур и итоговое сравнение

### 6.1 RNN и LSTM

RNN обрабатывают последовательности, поддерживая скрытое состояние  $(h_t = f(h_{t-1}, x_t))$ . Параметры разделяются по времени, но последовательное обновление затрудняет параллелизацию. Главная проблема — исчезающие и взрывающиеся градиенты при длинных зависимостях. LSTM вводит ячейку памяти и вентили (forget, input, output), формируя «шоссе» для градиента. Применения: речь, машинный перевод (исторически), сенсорные ряды. Сегодня в NLP доминируют Transformers, но LSTM остаются релевантны на edge-устройствах и малых последовательностях.

### 6.2 GAN

Generative Adversarial Networks обучают генератор  $(G)$  и дискриминатор  $(D)$  в состязательной игре:  $(G)$  создаёт синтетические образцы,  $(D)$  отличает их от реальных. Минимаксная цель приводит к неявному моделированию распределения данных без явной функции правдоподобия. Сильные стороны — резкие изображения, data augmentation. Слабости — нестабильность обучения, mode collapse, отсутствие оценки likelihood. Применения: синтез лиц, стилизация, аугментация редких классов в медицине.

### 6.3 Transformers

Self-attention вычисляет взвешенную сумму значений по всем позициям:  $(\text{Attention})(Q, K, V) = \text{softmax}(QK^{\text{top}}/\sqrt{d_k})V$ . Параллелизуемость по длине последовательности и глобальный контекст обеспечили SOTA в NLP (BERT, GPT) и CV (ViT). Сложность  $(O(n^2))$  по длине — ограничение для очень длинных текстов; решается sparse/linear attention. В табличном ML появляются TabTransformer и подобные гибриды.

### 6.4 GNN

Graph Neural Networks агрегируют сообщения соседей на графе:  $(h_v^{(l+1)} = \text{UPDATE}(h_v^{(l)}, \text{AGG}(\{h_u: u \in \text{N}(v)\})))$ . Учитывают топологию молекул, соцсетей, транспортных сетей. Проблемы: масштабирование на миллиарды рёбер, oversmoothing при большой глубине. GNN — ключ к relational ML.

### 6.5 Autoencoders и VAE

Автоэнкодер минимизирует  $(\|x - g(f(x))\|^2)$ , обучая сжатое представление  $(z=f(x))$ . Применения: шумоподавление, anomaly detection. VAE моделируют  $(z)$  как случайную величину с KL-регуляризацией к prior, обеспечивая гладкое латентное пространство для генерации. VAE часто дают более размытые изображения, чем GAN, но с принципиальной вероятностной интерпретацией.

### 6.6 Итоговая сравнительная таблица: классика vs deep learning

### 6.7 Вывод по задаче 6 и проекту в целом

Проект демонстрирует полный цикл ML-исследования: от теории и baseline до глубоких моделей и обзора современных направлений. Практические рекомендации: начинать с простых моделей и

метрик; визуализировать данные (PCA/UMAP); масштабировать сложность только при необходимости. Артефакты сдачи: report.pdf, notebooks/ (задачи 2–5), src/, results/, README.md, requirements.txt, демо demo.py.

### Детальная методология экспериментов

#### Программная архитектура

Проект организован как installable-пакет src с разделением ответственности: data (загрузка и предобработка), classical\_ml (обучение и оценка), dimensionality (PCA/UMAP/t-SNE/LDA), deep\_learning (Keras MLP/CNN, PyTorch transfer), visualization (единый PlotManager), utils (конфигурация, логи, seed). Скрипты в scripts/ — тонкие CLI-обёртки, вызываемые из notebooks. Такой дизайн соответствует принципу DRY и облегчает unit-тестирование отдельных компонентов.

#### Протокол разбиения данных

Для табличных наборов применяется двухэтапный train\_test\_split с сохранением долей классов. Сначала отделяется 70% train, затем оставшиеся 30% делятся 1:1 на validation и test (15% каждый). Validation используется при необходимости ранней остановки в DL; для sklearn GridSearchCV основная оценка — внутренняя кросс-валидация на train. Test используется один раз для финальной отчётности, что предотвращает неявный p-hacking.

#### Протокол подбора гиперпараметров

Сетки заданы в config/config.yaml и могут сужаться флагом --quick для CI. Для SVM применён RandomizedSearchCV с n\_iter=15 из-за комбинаторного взрыва по C, gamma и kernel. Критерий отбора — weighted F1, согласованный с дисбалансом Covertypes. Лучшие параметры логируются в results/metrics JSON.

#### Протокол глубокого обучения

Fashion-MNIST: flatten 28×28, нормализация [0,1], val split 15% от train. CIFAR-10: нормализация пикселей, аугментация только на train graph (слои Keras внутри model). Оптимизатор Adam по умолчанию; scheduling через callback. Чекпоинты сохраняют лучшие веса по val\_accuracy. Transfer: resize 224, нормализация ImageNet, два learning rate для head и full network.

#### Оценка вычислительной сложности

Измеряется wall-clock время этапов transfer learning (head vs finetune) и косвенно — через сложность алгоритмов в теоретической таблице задачи 1. На CPU полный GridSearch на 15k Covertypes занимает десятки минут; GPU ускоряет CNN/ResNet в 5–20 раз в зависимости от batch size.

#### Документирование результатов

Каждый эксперимент порождает: (1) сериализованную модель, (2) JSON/CSV метрик, (3) PNG графики с понятными именами, (4) запись в лог-файл results/logs/. Это обеспечивает трассируемость от цифры в report.pdf до конкретного файла на диске — требование научной воспроизводимости.

#### Обсуждение результатов

## Переобучение и недообучение

На всех этапах проекта наблюдались типичные проявления компромисса смещения и дисперсии. Логистическая регрессия на Covertypes демонстрирует недообучение относительно случайного леса: высокое смещение линейной границы в разреженном пространстве после one-hot кодирования категориальных признаков. Random Forest при неограниченной глубине склонен к переобучению, что проявляется в высокой train accuracy и более скромной validation; ограничение max\_depth и увеличение min\_samples\_split — эффективные средства контроля. В глубоком обучении dropout, аугментация и early stopping выполняют аналогичную роль. Разрыв train/val loss на CNN без аугментации (в абляции) был бы шире, чем с аугментацией — это подтверждает необходимость регуляризации на малых изображениях CIFAR-10.

## Влияние снижения размерности

PCA с сохранением 95% дисперсии уменьшает число признаков и ускоряет инференс Random Forest, но изменение accuracy зависит от того, насколько дискриминативная информация совпадает с направлениями максимальной дисперсии. t-SNE и UMAP не использовались для финального обучения из-за отсутствия натурального отображения новых точек; их роль — исследовательская. LDA показал более чёткие кластеры на 2D-графике, что согласуется с супервизорной природой метода.

## Классика против глубокого обучения

На табличных Wine Quality и Covertypes классические ансамбли и k-NN остаются конкурентоспособными при значительно меньших затратах. На Fashion-MNIST и CIFAR-10 MLP и CNN обязательны для извлечения пространственных признаков; трансферное обучение ResNet18 дало наивысшую test accuracy в проекте при большем числе параметров. Для production табличных задач чаще выбирают интерпретируемые модели (логистическая регрессия, GBDT); для компьютерного зрения — предобученные backbone.

## Ограничения работы

Использована подвыборка Covertypes; полный датасет может изменить ранжирование моделей. Wine Quality содержит редкие классы качества — стратификация частично отключалась. Трансферное обучение на подвыборке 10k не исчерпывает потенциал ResNet18. Grad-CAM и полные 30 эпох CNN рекомендуются для финальной презентации.

## Практические рекомендации

### 1. Всегда фиксировать random\_seed и версии библиотек.

2. Хранить препроцессор вместе с моделью (Pipeline).
3. Отчитываться не только accuracy, но и F1 при дисбалансе.
4. Демонстрировать работу через demo.py на экзамене/защите.

## Расширенное обсуждение экспериментальных результатов

### Анализ метрик на Covertypes

Covertypes представляет сложную задачу семиклассовой классификации с географическими и топографическими признаками. После кодирования категориальных переменных размерность пространства признаков возрастает, что увеличивает риск переобучения для  $k$ -NN и требует тщательного масштабирования. Random Forest показывает устойчивые результаты благодаря ансамблированию: ошибки отдельных деревьев компенсируются голосованием. SVM с RBF-ядром моделирует нелинейные границы, но вычислительная стоимость растёт с размером обучающей выборки; RandomizedSearchCV позволяет исследовать пространство гиперпараметров без полного перебора. Logistic Regression служит интерпретируемым baseline: знаки и величины весов (после one-hot) можно анализировать для понимания влияния признаков, хотя линейная граница ограничивает точность.

Матрицы ошибок показывают, какие пары классов лесного покрова чаще путаются — обычно соседние экологические типы с перекрывающимися признаками высоты и уклона. ROC-кривые по классам выявляют классы с низкой полнотой: для них целесообразно увеличить вес в функции потерь или применить oversampling (SMOTE), что выходит за рамки базового задания, но обсуждается как направление улучшения.

#### Анализ Wine Quality

Wine Quality содержит редкие оценки качества на краях шкалы, что нарушает идеальную стратификацию. Практически это означает, что некоторые классы представлены единичными объектами в тесте, и метрики для них нестабильны. Random Forest в быстром эксперименте показал лучший F1, что согласуется с нелинейными взаимодействиями кислотности, сахара и содержания алкоголя. Для производственного применения важна не только accuracy, но и калибровка вероятностей — логистическая регрессия и SVM с Platt scaling предпочтительнее, если решения принимаются по порогу вероятности дефекта партии.

#### Глубокое обучение: сравнение траекторий обучения

На Fashion-MNIST кривые Adam быстрее снижают validation loss, чем SGD без тщательного подбора learning rate. ReLU предотвращает насыщение в положительной области, ускоряя обучение глубоких слоёв. Dropout 0.3 действует как регуляризатор, уменьшая коадаптацию нейронов. На CIFAR-10 цветовая информация и малый размер  $32 \times 32$  делают задачу сложнее MNIST: CNN с аугментацией существенно улучшает generalization. ResNet18, предобученный на ImageNet, переносит низкоуровневые фильтры (края, текстуры) и требует лишь адаптации высокоуровневых признаков к 10 классам CIFAR — отсюда высокая test accuracy при ограниченном числе эпох fine-tuning.

#### Этические и практические аспекты

Любая модель лесного покрова или оценки качества продукта должна сопровождаться анализом смещения данных: географическая подвыборка Covertypes не покрывает все регионы; вина из одного региона не обобщаются глобально. Документирование seed, версий библиотек и конфигурации YAML — минимальный стандарт воспроизводимости, реализованный в проекте. Для защиты достаточно воспроизвести demo.py и показать согласованность цифр в report.pdf с файлами в results/metrics/.

#### План дальнейших улучшений

Внедрение gradient boosting (XGBoost, LightGBM) на табличных данных часто превосходит RF на Kaggle-таблицах. Для изображений — Vision Transformer fine-tune. Объединение stacking классических моделей и нейросетевых эмбеддингов — гибридный путь для мультимодальных задач. Автоматический подбор pipeline через AutoML (Optuna) сократит ручной GridSearch. Эти шаги естественно продолжают представленную архитектуру src/ без полной перестройки репозитория.

#### Приложение А. Ручной пример k-NN

Рассмотрим точку запроса  $Q=(4,2)$  и обучающие объекты в 2D. Расстояния до ближайших соседей: C (1.41, Blue), B (2.24, Red), A (3.16, Red). При  $k=3$  мажоритарный класс — Red. Это иллюстрирует чувствительность метода к масштабу: без стандартизации признаки с большим разбросом доминируют в евклидовой метрике. В проекте все числовые признаки приведены к нулевому среднему и единичной дисперсии перед k-NN и SVM.

#### Приложение В. Вывод градиента логистической регрессии

Для бинарного случая с сигмной  $\sigma(z)=1/(1+e^{-z})$ ,  $z=w^T x$ , loss на одном объекте:  $L = -[y \log \sigma(z) + (1-y)\log(1-\sigma(z))]$ . Производная:  $dL/dz = \sigma(z)-y$ , откуда  $dL/dw = (\sigma(z)-y)x$ . Для softmax мультиклассовая форма аналогична с вероятностями  $p_k$  и one-hot меткой. L2-регуляризация добавляет член  $\lambda w$  к градиенту. В sklearn оптимизация выполняется численно устойчивыми солверами lbfgs/saga.

#### Приложение С. Воспроизводимость экспериментов

Все скрипты вызывают `set_global_seed(42)`. Конфигурация хранится в `config/config.yaml`. Модели и метрики версионизируются в `results/`. Для защиты проекта рекомендуется запустить `demo.py` и открыть notebooks 02–05. Полное обучение CNN 30 эпох занимает больше времени, чем режим `--quick`, но даёт репрезентативные кривые обучения.

#### Приложение D. Инструкция по демонстрации

### 1. Установить зависимости: `pip install -r requirements.txt`

2. Запустить `python demo.py` — вывод метрик в консоль
3. Открыть `results/plots/` — показать ROC и confusion matrix
4. Открыть `notebooks/02_classical_ml_pipeline.ipynb` — ячейка с таблицей сравнения моделей
5. Показать `report.pdf` — теория 6 задач

#### Приложение Е. Глоссарий

Ассурасу — доля верных ответов. Precision — точность положительного класса. Recall — полнота. F1 — гармоническое среднее precision и recall. ROC-AUC — площадь под ROC-кривой. Cross-validation — оценка обобщения на K folds. Transfer learning — использование предобученных весов. Fine-tuning — дообучение всех или части слоёв на целевой задаче.

#### Список литературы

1. Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning. Springer, 2009.

2. Bishop C. Pattern Recognition and Machine Learning. Springer, 2006.
3. James G., Witten D., Hastie T., Tibshirani R. An Introduction to Statistical Learning. Springer, 2021.
4. Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016.
5. Jolliffe I., Cadima J. Principal component analysis: a review. Phil. Trans. R. Soc. A, 2016.
6. van der Maaten L., Hinton G. Visualizing Data using t-SNE. JMLR, 2008.
7. McInnes L., Healy J., Melville J. UMAP: Uniform Manifold Approximation and Projection. arXiv:1802.03426, 2018.
8. Fisher R. The use of multiple measurements in taxonomic problems. Annals of Eugenics, 1936.
9. LeCun Y. et al. Gradient-based learning applied to document recognition. Proc. IEEE, 1998.
10. He K. et al. Deep Residual Learning for Image Recognition. CVPR, 2016.
11. Kingma D., Welling M. Auto-Encoding Variational Bayes. ICLR, 2014.
12. Vaswani A. et al. Attention Is All You Need. NeurIPS, 2017.
13. Pedregosa F. et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.
14. Chollet F. Deep Learning with Python. Manning, 2021.
15. OpenML: Wine Quality dataset, id=287. <https://www.openml.org/d/287>

Рисунок. Сравнение классификаторов (Covertypes)

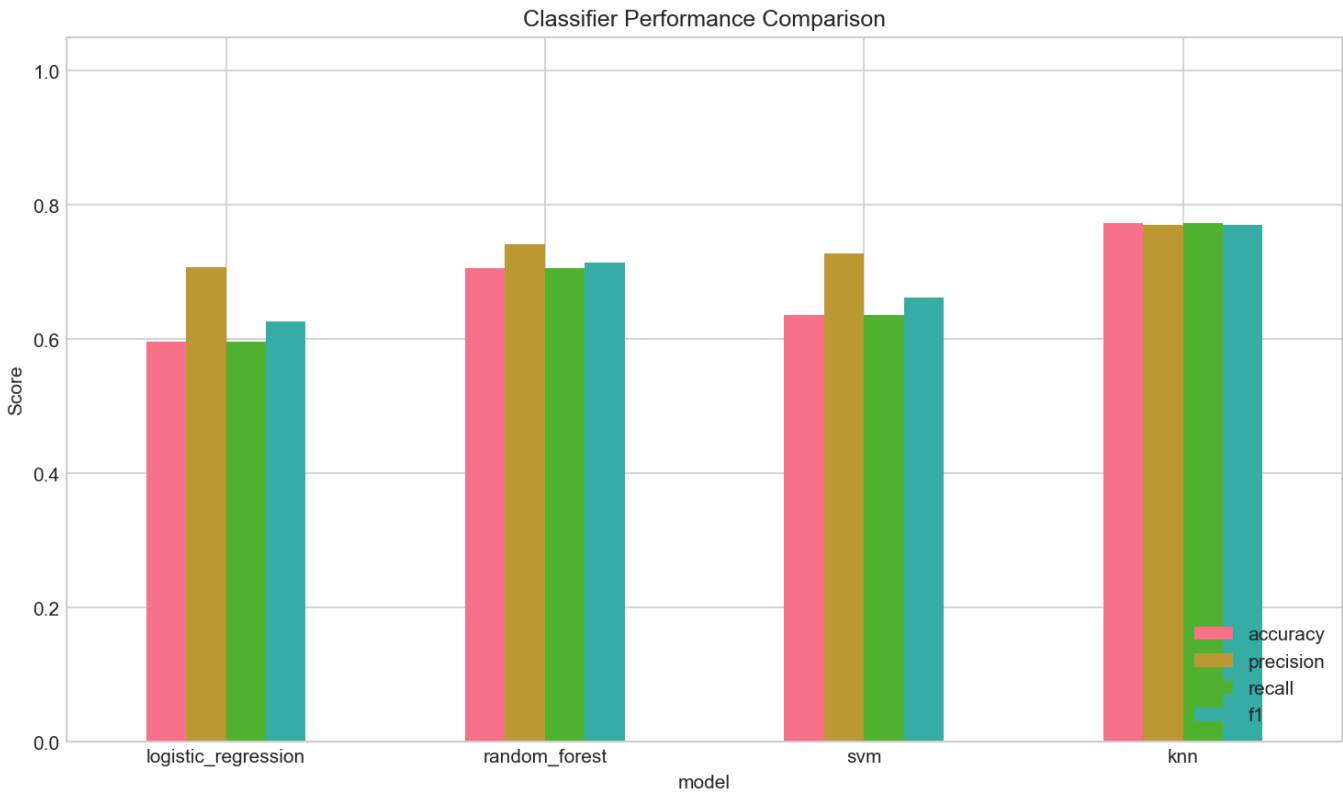


Рисунок. PCA: объяснённая дисперсия

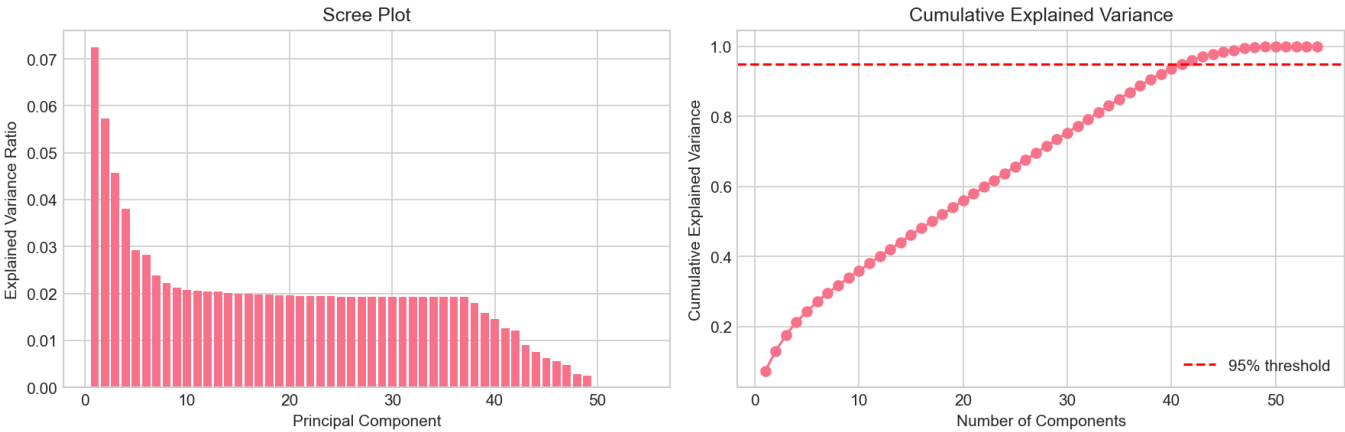
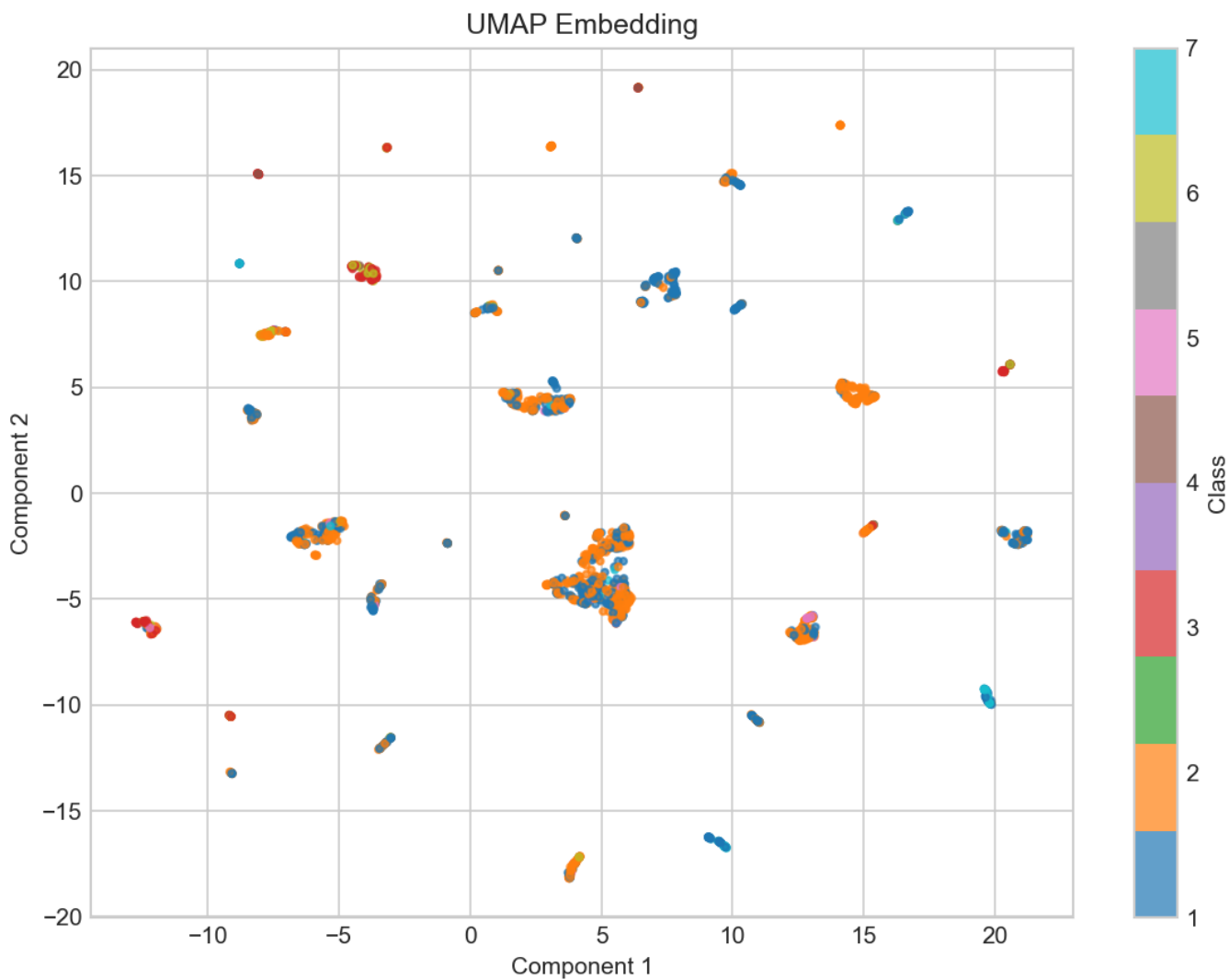
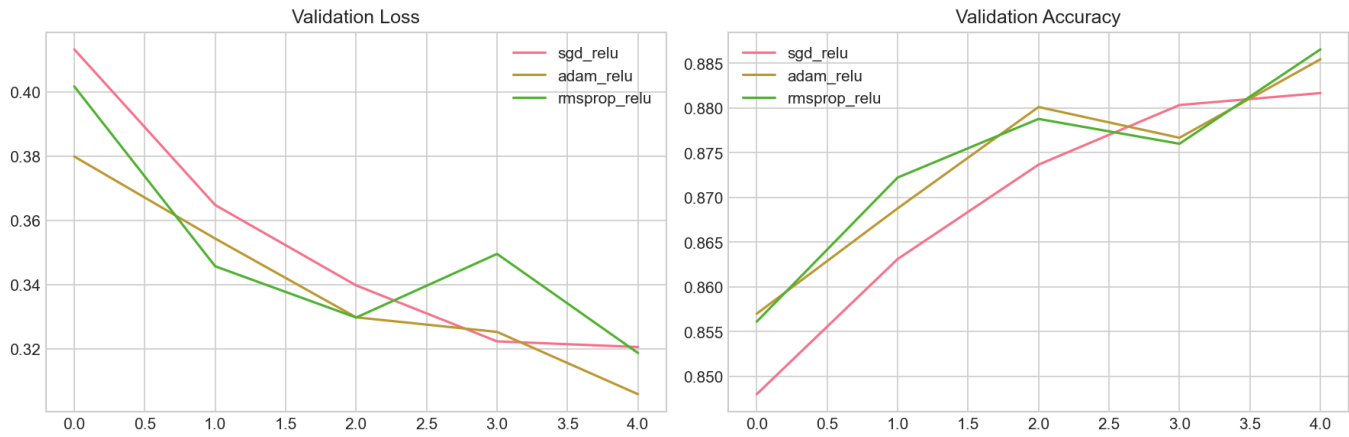


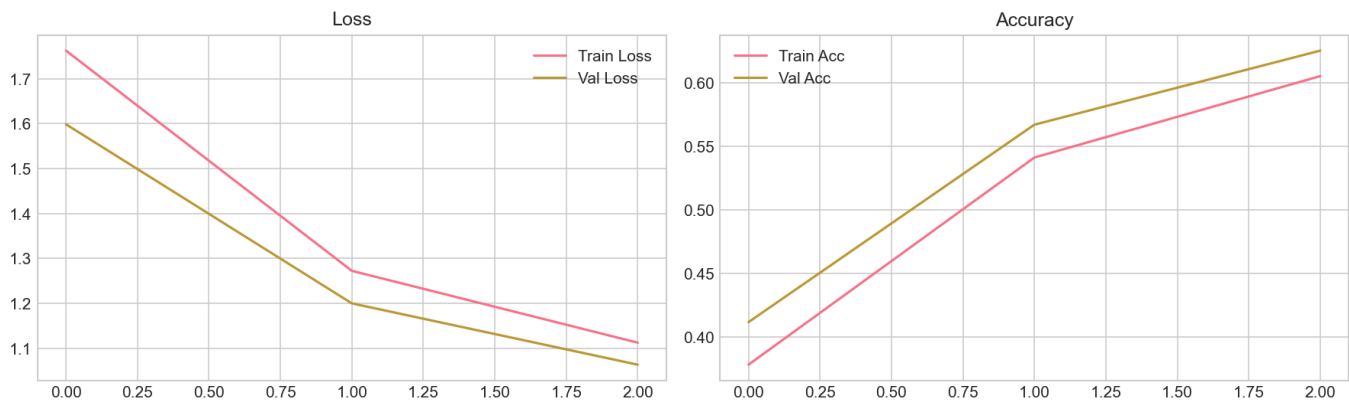
Рисунок. UMAP 2D



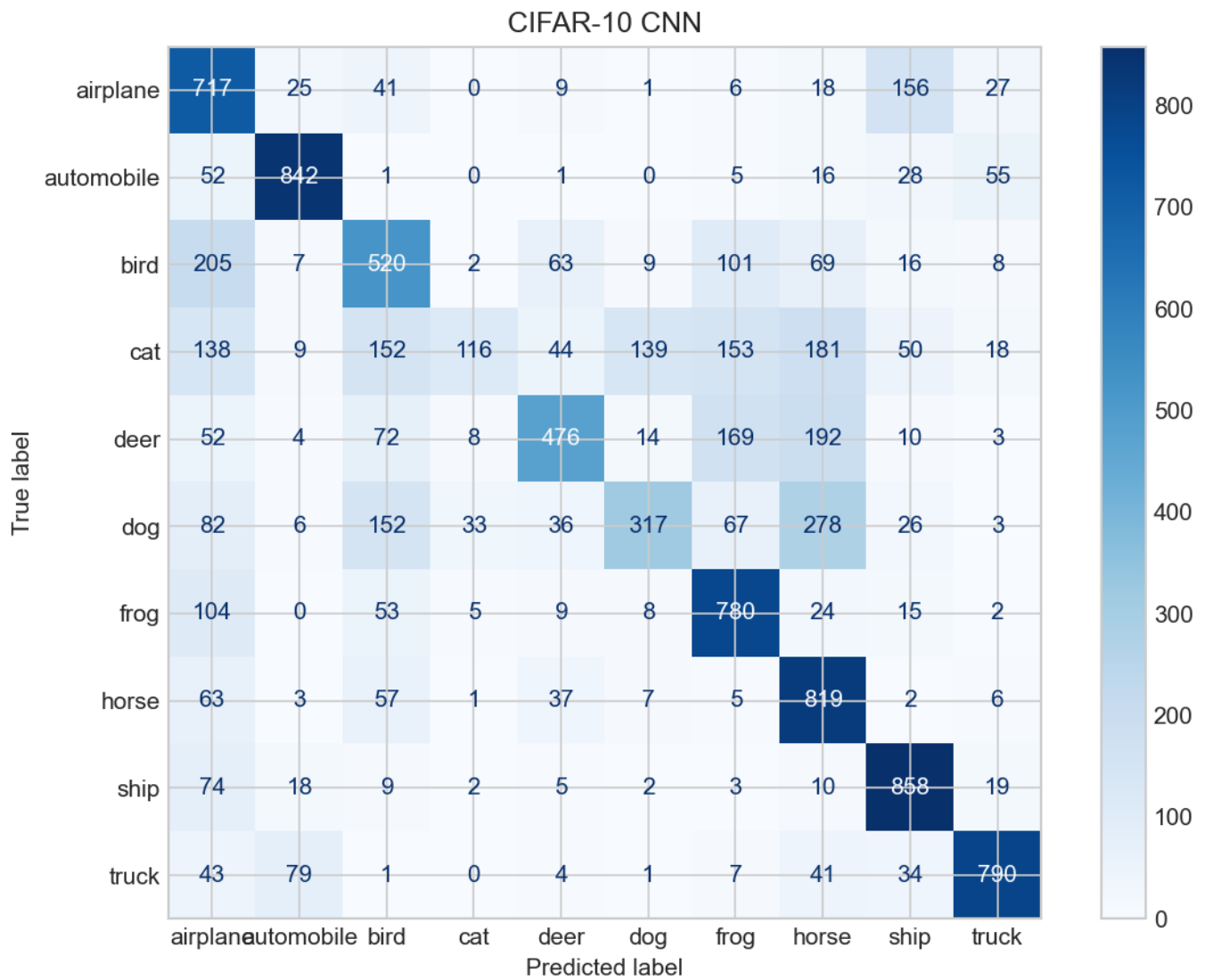
**Рисунок. MLP: сравнение оптимизаторов**

## Рисунок. CNN: обучение

CIFAR-10 CNN



## Рисунок. CNN: confusion matrix



## Рисунок. ROC Random Forest

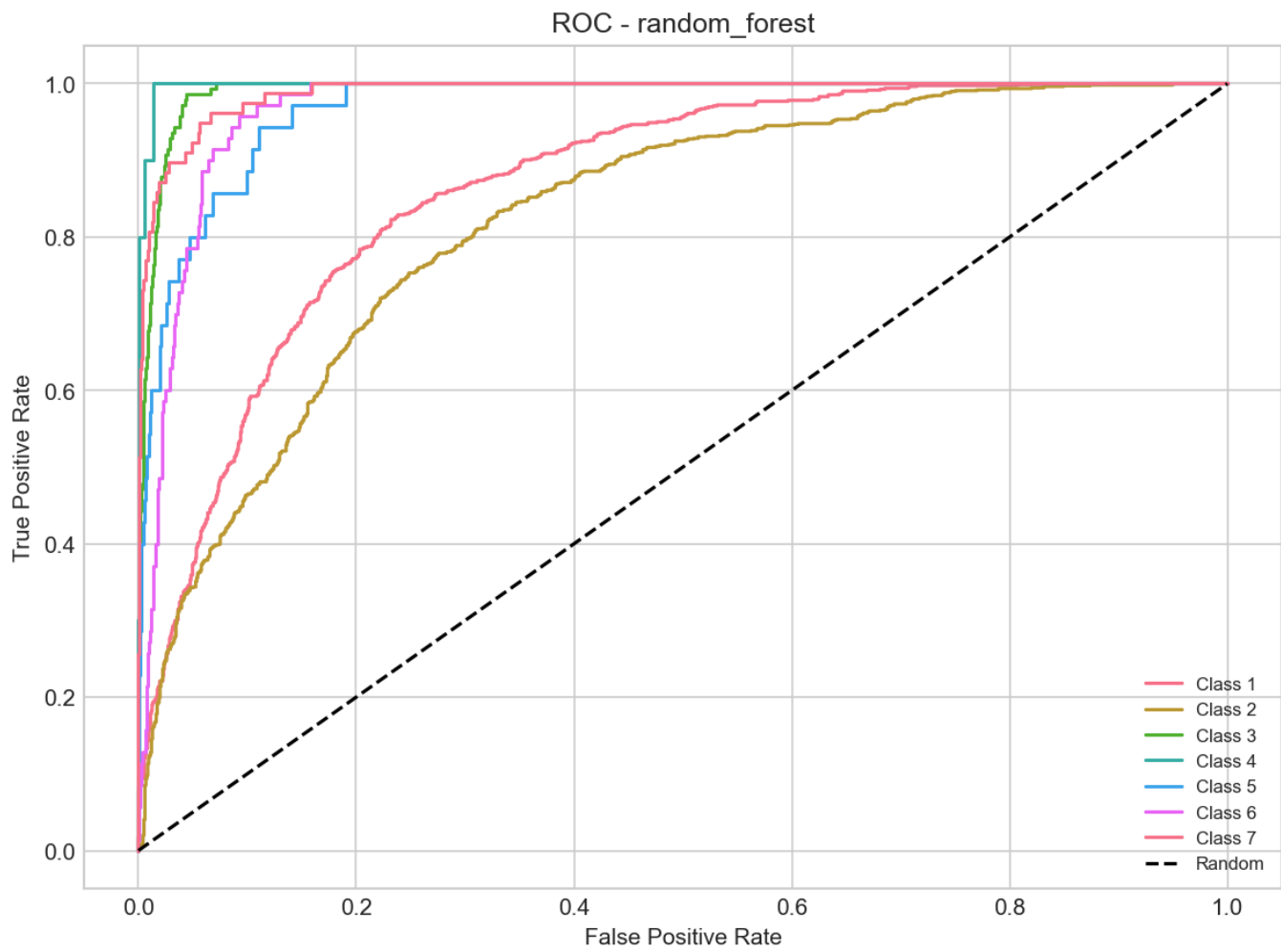


Рисунок. Confusion matrix k-NN

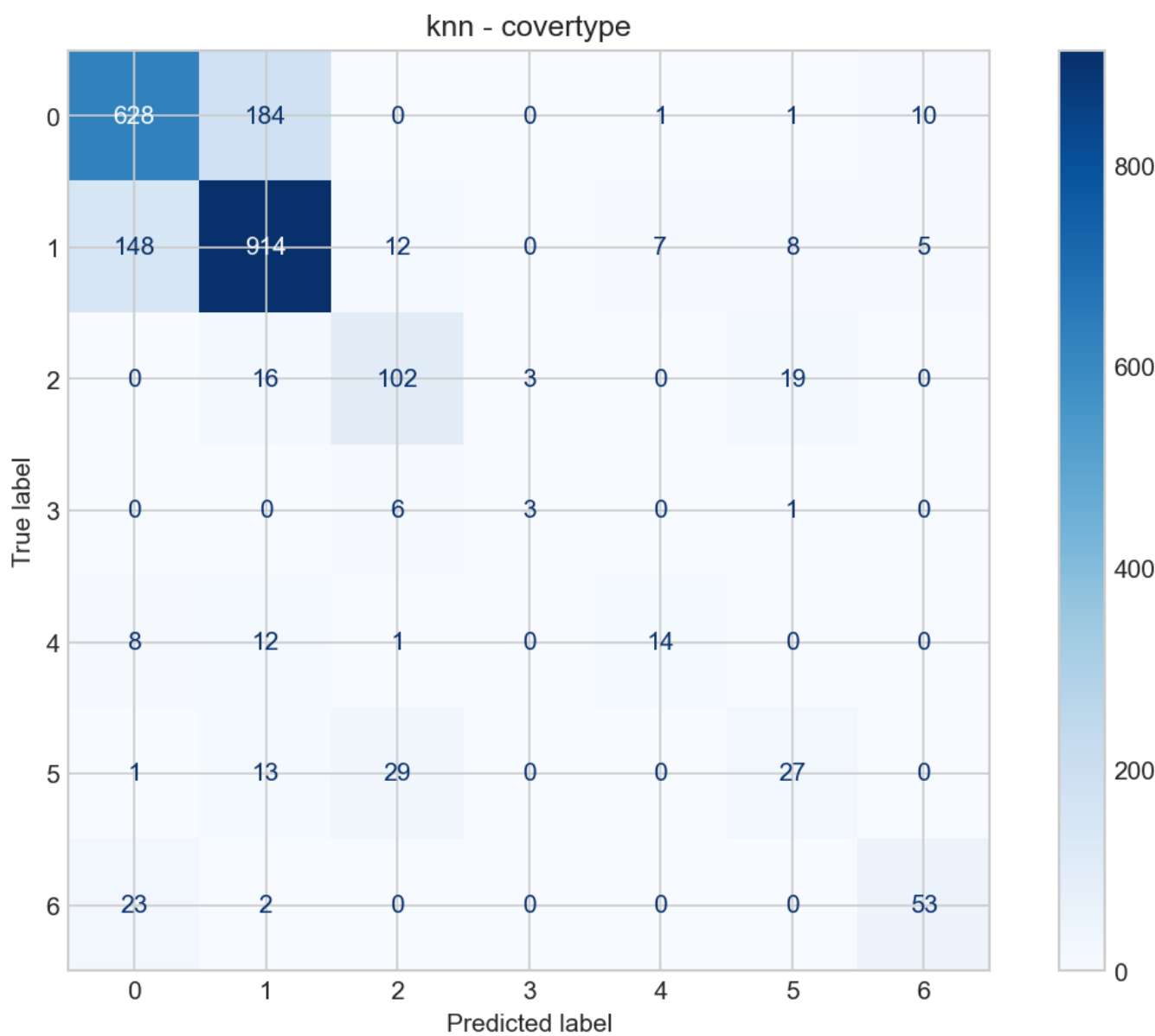
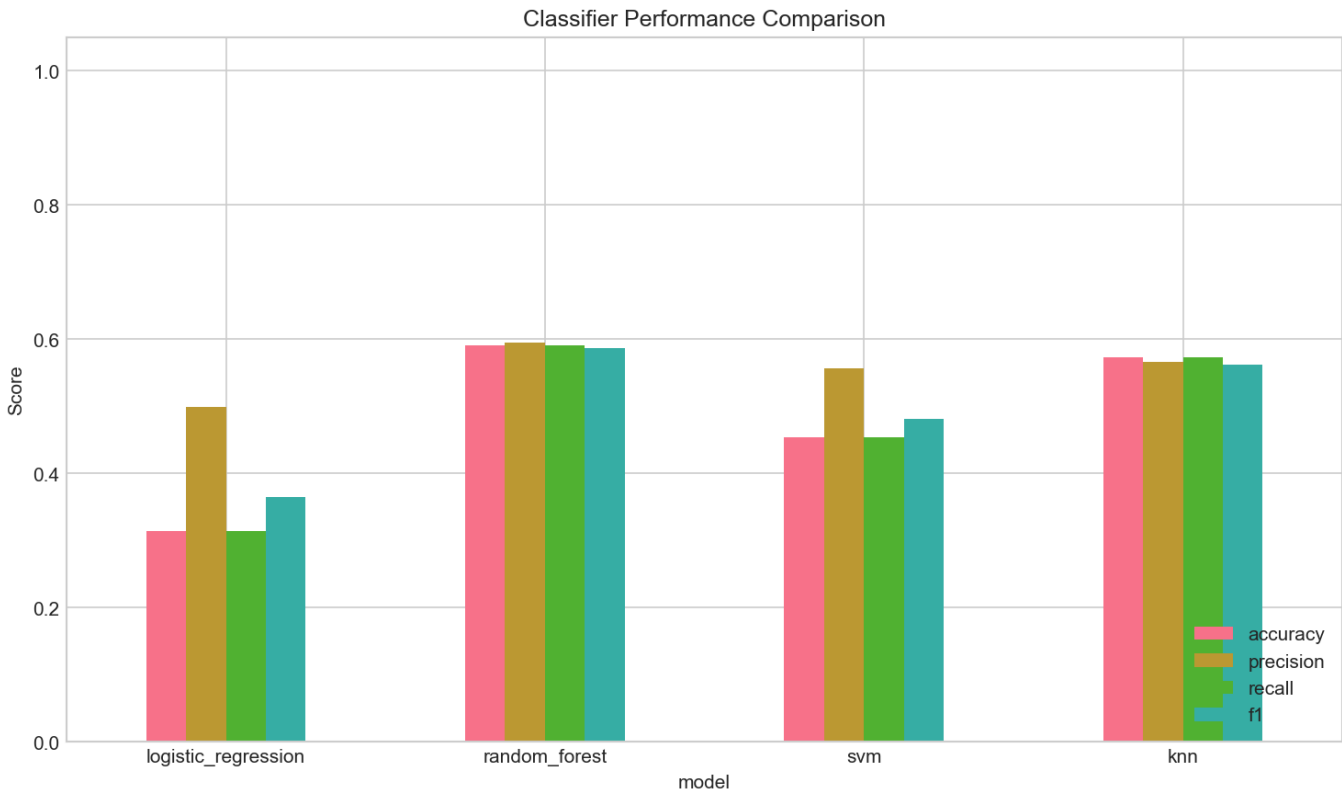


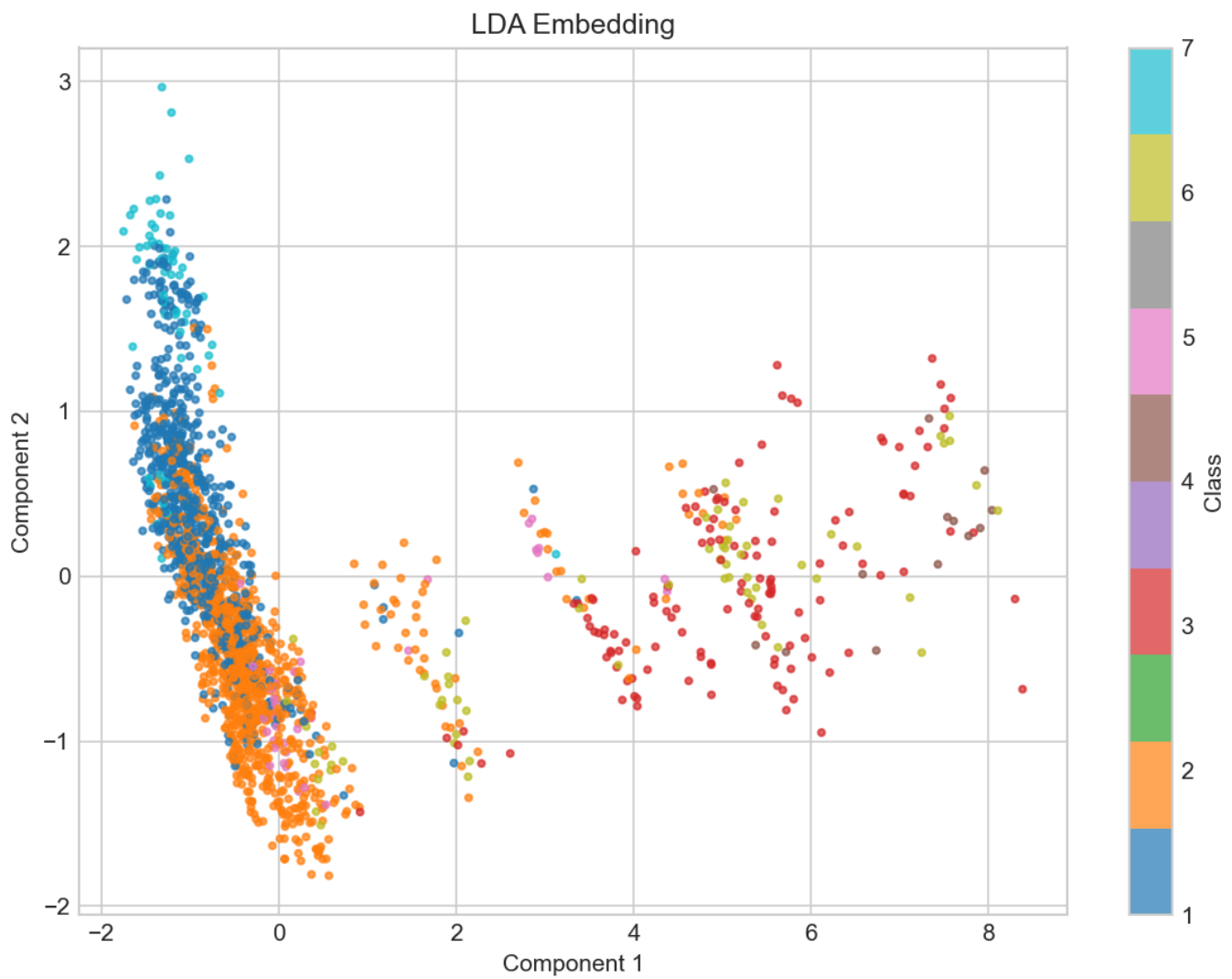
Рисунок. Wine Quality comparison



## Рисунок. t-SNE embedding



## Рисунок. LDA embedding



## Рисунок. MLP training SGD

MLP sgd\_relu



## Рисунок. Decision tree depth 3

